

# **PCAD**

## **Programmazione Concorrente**

### **Algoritmi Distribuiti**

**Arnaud Sangnier**  
arnaud.sangnier@unige.it

**Sistemi distribuiti con fallimenti**

# Modelli di sistemi distribuiti

- **Memoria condivisa:**
  - I processi comunicano tramite scrittura/lettura nella memoria condivisa
  - Modello classico per studiare problemi di coordinamento
  - Utile per modellare programmi multi-processor o multi-threaded che si eseguono su un unico computer
  - Astrazione conveniente per la programmazione
- **Message passing:**
  - I processi/nodi comunicano scambiandosi messaggi
  - La topologia di comunicazione può essere completamente connessa o un grafo arbitrario
  - Utile per modellare sistemi distribuiti su vari siti
- **Memoria condivisa vs Message passing:**
  - Esistono delle simulazioni di un modello nell'altro

**In questa lezione => Message passing**

# Sincronizzazione

- **Sistemi sincronizzati:**

- I nodi si eseguono in tappe sincronizzate (chiamate **round**)
  - Il tempo è discreto:  $0, 1, 2, 3, \dots$
  - Il round  $r$  avviene fra il tempo  $r-1$  e il tempo  $r$
- Al tempo  $r-1$ , ciascun processo invia messaggi (oppure un unico messaggio) che verranno ricevuti e elaborati al tempo  $r$

- **Sistemi asincroni:**

- La velocità di ciascun processo è arbitraria (ma finita) come il tempo di trasmissione dei messaggi
- I messaggi vengono sempre trasmessi (nell'ordine dell'emissione) **a meno che non si verifichino degli errori**

# Fallimenti

- **Crash failure:**
  - Un nodo/processo può smettere di funzionare ad un momento dell'esecuzione
  - Questo può succedere in mezzo ad un round (per gli sistemi sincronizzati)
    - Alcuni messaggi del round possono essere stati inviati e altri no
- **Byzantine failure:**
  - Un nodo/processo inizia a comportarsi in un modo completamente arbitrario
  - Diversi nodi bizantini possono complottare
- **Omission failure**
  - Nodi/processi/link di comunicazione smettono di funzionare temporaneamente
  - Ad esempio alcuni messaggi possono essere persi

# Modello formale per message passing

**Modello semplice ma necessario per potere ottenere risultati formali**

- **Modello basico:**

- Un sistema è composto da  $n$  nodi deterministici  $v_1, \dots, v_n$  e da canali di comunicazione fra ciascun paio di nodi
  - Si assume che la numerazione dei nodi da 1 a  $n$  è conosciuta (ciascun nodo conosce il proprio numero)
- In ogni istante, un nodo  $v_i$  è in uno stato interno  $Q_i$
- Il cambiamento dello stato del sistema è dovuto a eventi discreti (tipicamente l'emissione e la ricezione di messaggi) → Modello event based

# Modello event based

- **Stati interni dei nodi**

- Inputs (valori iniziali), variabili locali, avvolte orologi locali
- Storia di tutti gli eventi osservati

- **Tipo di eventi**

- **Send event**: un nodo  $v_i$  invia un messaggio sul canale di comunicazione per un altro node  $v_j$
- **Received event**: un nodo  $v_j$  riceve un messaggio
  - Deve essere preceduto da un send event corrispondente
- **Timing event**: evento visto da un nodo da un orologio locale (meno rilevante in questa lezione)

# Schedule e esecuzioni

- **Configurazione C:** vettore di tutti gli  $n$  stati dei nodi (in un dato momento)
  - Una configurazione = lo stato del sistema in un determinato istante
- **Fragmento di esecuzione:**
  - sequenza alternata di configurazioni ed eventi
  - Esempio:  $C_0 E_1 C_1 E_2 C_2 E_3 C_3 E_4 \dots$ 
    - $C_i$  sono le configurazioni e  $E_i$  sono gli eventi
  - Ogni triplo  $C_i E_{(i+1)} C_{(i+1)}$  deve essere coerente con le regole indotte dall'evento  $E_{(i+1)}$ 
    - Ad esempio se  $E_{(i+1)}$  è un evento di ricezione dal nodo  $v_j$  allora l'evento modifica solo lo stato del nodo  $v_j$
- **Esecuzione:** frammento di esecuzione che inizia dalla configurazione iniziale
- **Schedule:** esecuzione senza le configurazioni

# Modello message passing: commenti

- **Stati locali:**
  - In uno stato di un nodo  $v_j$  non sono presenti informazioni sullo stato dei messaggi inviati (il nodo non può sapere se il suo messaggio è stato ricevuto o meno)
- **Nodi deterministici:**
  - Supponiamo che i nodi siano deterministici
    - Data una sequenza di eventi relativi a un nodo, il nodo si comporterà sempre nello stesso modo
  - Esistono dei modelli in cui i nodi possono avere un comportamento randomizzato o casuale (per esempio lanciando una moneta)



# Local Schedules

**Lo stato di un nodo  $v$  è determinato dai suoi valori di input e dagli eventi che osserva**

- **Restrizione di schedule:**
  - Dato uno schedule  $S$ , la restrizione  $S|_i$  è la sotto sequenza di  $S$  costituita dagli input del nodo  $v_i$  e da tutti gli eventi che succedono al nodo  $v_i$  (eventi che questo nodo osserva)
- **Esempio:**
  - 3 nodi  $v_1$ ,  $v_2$  e  $v_3$ , Eventi  $s_{ij}$  (nodo  $v_i$  manda a nodo  $v_j$ ) e  $r_{ji}$  (nodo  $v_j$  riceve da nodo  $v_i$ )
  - Schedule:  $S = s_{13} s_{23} s_{31} r_{13} s_{32} r_{31} r_{23} s_{13} s_{21} r_{31} r_{12} r_{32}$
  - $S|_1 = s_{13} r_{13} s_{13} r_{12}$
  - $S|_2 = s_{23} r_{23} s_{21}$
  - $S|_3 = s_{31} s_{32} r_{31} r_{31} r_{32}$

# Indistinguibilità

- **Teorema (Indistinguibilità):**

- Se abbiamo due schedule  $S$  e  $S'$  e un nodo  $v_i$  che ha gli stessi input in  $S$  e  $S'$  e tale che  $S|i = S'|i$ , allora se  $v_i$  esegue la prossima azione, eseguirà la stessa azione sia in  $S$  che in  $S'$ .

- **Prova:**

- Lo stato di dipende solo dai suoi inputs e da  $S|i$
- Quando il comportamento è deterministico, la prossima azione dipende unicamente dallo stato
- L'indistinguibilità viene utilizzata per dimostrare l'efficienza di alcuni algoritmi e soprattutto alcuni risultati di **impossibilità**

# Il problema dei due generali

- Immaginiamo due generali dello stesso esercito che vogliono attaccare un nemico.
- I due generali (e i loro soldati) si trovano su due montagne mentre il nemico si trova in mezzo alla valle
- Devono mettersi d'accordo sull'ora in cui attaccare, perché se non attaccano insieme perdono
- La comunicazione avviene tramite piccioni viaggiatori che attraversano la valle per recapitare i messaggi
- **Problema:** i piccioni potrebbero non riuscire ad attraversare la valle

**I messaggi potrebbero andare persi**

# Il problema dei due generali

- Il problema ha senso nel mondo reale
  - Alice e Bob vogliono uscire sabato sera
    - Devono mettersi d'accordo su:
      - Dove e quando incontrarsi
      - Chi prenota per la cena
      - etc
    - Possono solo comunicare via un servizio di messaggistica che non garantisce la ricezione dei messaggi
- In una rete distribuita, i nodi devono mettersi d'accordo su;
  - Un leader per eseguire un calcolo
  - l'ordine in cui fare degli accessi conflittuali ai dati
  - Se accettare una transazione su un database distribuito
  - ecc

# Il problema dei due generali formalmente

- **Modello:** due nodi deterministici, comunicazione sincronizzata, possibile perdita dei messaggi
- **Input:** ciascun nodo inizia con uno delle due input possibili 0 o 1
  - Gli input codificano l'ora dell'attacco
- **Output:** Ciascun nodo deve decidere un valore fra 0 e 1
- Proprietà da verificare:
  - **Agreement:** i due nodi devono decidere lo stesso valore
  - **Validity:** se i due nodi hanno lo stesso valore  $x$  in  $\{0,1\}$  e i messaggi non sono persi, i due nodi devono decidere  $x$ 
    - Se i nodi iniziano con dei valori diversi o se dei messaggi sono persi, i nodi possono decidere 0 o 1 dal momento che decidono lo stesso valore
  - **Termination:** i due nodi devono decidere dopo un numero finito di round

# Impossibilità per i due generali

- **Prova di indistinguibilità:**

- Una esecuzione  $E$  è indistinguibile da una esecuzione  $E'$  per un nodo  $v$  se  $v$  vede le stesse cose nelle due esecuzioni
  - Stessi input e messaggi mandati e ricevuti
- Se  $E$  è indistinguibile da  $E'$  per  $v$  allora  $v$  fa le stesse cose nelle due esecuzioni (in particolare decide lo stesso output)
  - Scriviamo allora  $E|_v = E'|_v$

- **Similarità:**

- Prendiamo tutte le esecuzioni possibile  $E_1, E_2, \dots$
- Diciamo che  $E_i$  e  $E_j$  sono simili per il nodo  $v$  se  $E_i|_v = E_j|_v$

$$E_i \sim_v E_j \text{ sse } E_i|_v = E_j|_v$$

# Impossibilità per i due generali

- Se prendiamo una sequenza di esecuzione  $E_0, E_1, \dots, E_k$  tale che per tutti  $i$  in  $\{1, \dots, k\}$   $E_{i-1}$  e  $E_i$  sono simili per un nodo  $v_i$

$$E_0 \sim_{v_1} E_1 \sim_{v_2} E_2 \sim_{v_3} E_3 \sim_{v_4} E_4 \sim_{v_5} E_5 \dots$$

- Allora come abbiamo  $E_{i-1}|v_i = E_i|v_i$ , il nodo  $v_i$  produce lo stesso output in  $E_{i-1}$  e  $E_i$
- Ma in più se abbiamo l'agreement, i.e. tutti i nodi devono produrre lo stesso output allora tutti i nodi producono lo stesso output in  $E_0, E_1, \dots, E_k$ .
  - In  $E_0$  e  $E_1$  tutti i nodi producono lo stesso valore di  $v_1$  che produce lo stesso valore nelle due esecuzione,
  - In  $E_1$  e  $E_2$  tutti i nodi producono lo stesso valore di  $v_2$  che produce lo stesso valore nelle due esecuzione,
  - ....
- **La prova di impossibilità costruisce una tale sequenza portandoci ad una contraddizione**

# Impossibilità per i due generali: prova

- Supponiamo che c'è un  $T$ -round protocol
  - I nodi devono decidere dopo esattamente  $T$  rounds
  - Costruiamo una sequenza di esecuzioni  $E_0, E_1, \dots, E_{4T+2}$  tale che:
    - Esecuzione  $E_0$  : due input sono 0 e nessun messaggio viene perso
      - L'output dei due nodi deve dunque essere 0
    - Esecuzione  $E_{4T+2}$  : due input sono 1 e nessun messaggio viene perso
      - L'output dei due nodi deve dunque essere 1
    - Per tutti  $i$  in  $\{1, \dots, 4T+2\}$ , abbiamo  $E_{i-1} \sim_v E_i$  per un nodo  $v$  in  $\{v_1, v_2\}$

**Se riusciamo avremo una contraddizione con quello detto nel lucido precedente**



# Impossibilità per i due generali: prova

**I nodi decidono esattamente dopo  $T$  rounds e possiamo assumere che in ogni round ciascun nodo invii un messaggio all'altro nodo**

- Esecuzione  $E_0$  : due input sono 0 e nessun messaggio viene perso
- Esecuzione  $E_1$  : due input sono 0 e l'ultimo messaggio di  $E_0$  ricevuto viene perso
- Esecuzione  $E_2$  : due input sono 0 e l'ultimo messaggio di  $E_1$  ricevuto viene perso
- ...
- Esecuzione  $E_{2T}$  : due input sono 0 e tutti i messaggi vengono persi
  - In  $E_{2T}$ , grazie alla catena di similarità i due nodi decidono 0 come in  $E_0$

# Impossibilità per i due generali: prova

- Esecuzione  $E_{4T+2}$  : due input sono 1 e nessun messaggio viene perso
- Esecuzione  $E_{4T+1}$  : due input sono 1 e l'ultimo messaggio di  $E_{4T+2}$  ricevuto viene perso
- Esecuzione  $E_{4T}$  : due input sono 1 e l'ultimo messaggio di  $E_{4T+1}$  ricevuto viene perso
- ...
- Esecuzione  $E_{2T+2}$  : due input sono 1 e tutti i messaggi vengono persi
  - In  $E_{2T+2}$ , grazie alla catena di similarità i due nodi decidono 1 come in  $E_{4T+2}$

# Impossibilità per i due generali: prova

- Esecuzione  $E_{2T}$  : due input sono 0 e tutti i messaggi vengono persi
  - Output dei due nodi sono 0
- **Esecuzione  $E_{2T+1}$  : v1 ha input 0 e v2 ha input 1 e tutti i messaggi vengono persi**
  - **Per similarità con  $E_{2T}$  , l'output dei due nodi deve essere 0 e per similarità con  $E_{2T+2}$ , l'output dei due nodi deve essere 1 => CONTRADIZIONE**
- Esecuzione  $E_{2T+2}$  : due input sono 1 e tutti i messaggi vengono persi
  - Output dei due nodi sono 1